

Mô hình hoá phần mềm

C4 Model

C4 Model là gì?

C4 Model là một framework giúp trực quan hóa kiến trúc phần mềm, được tạo ra bởi **Simon Brown**.

 Bản đồ cho mã nguồn, cung cấp các mức độ chi tiết (zoom levels) khác nhau:

 **Level 1: System Context**

 **Level 2: Container**

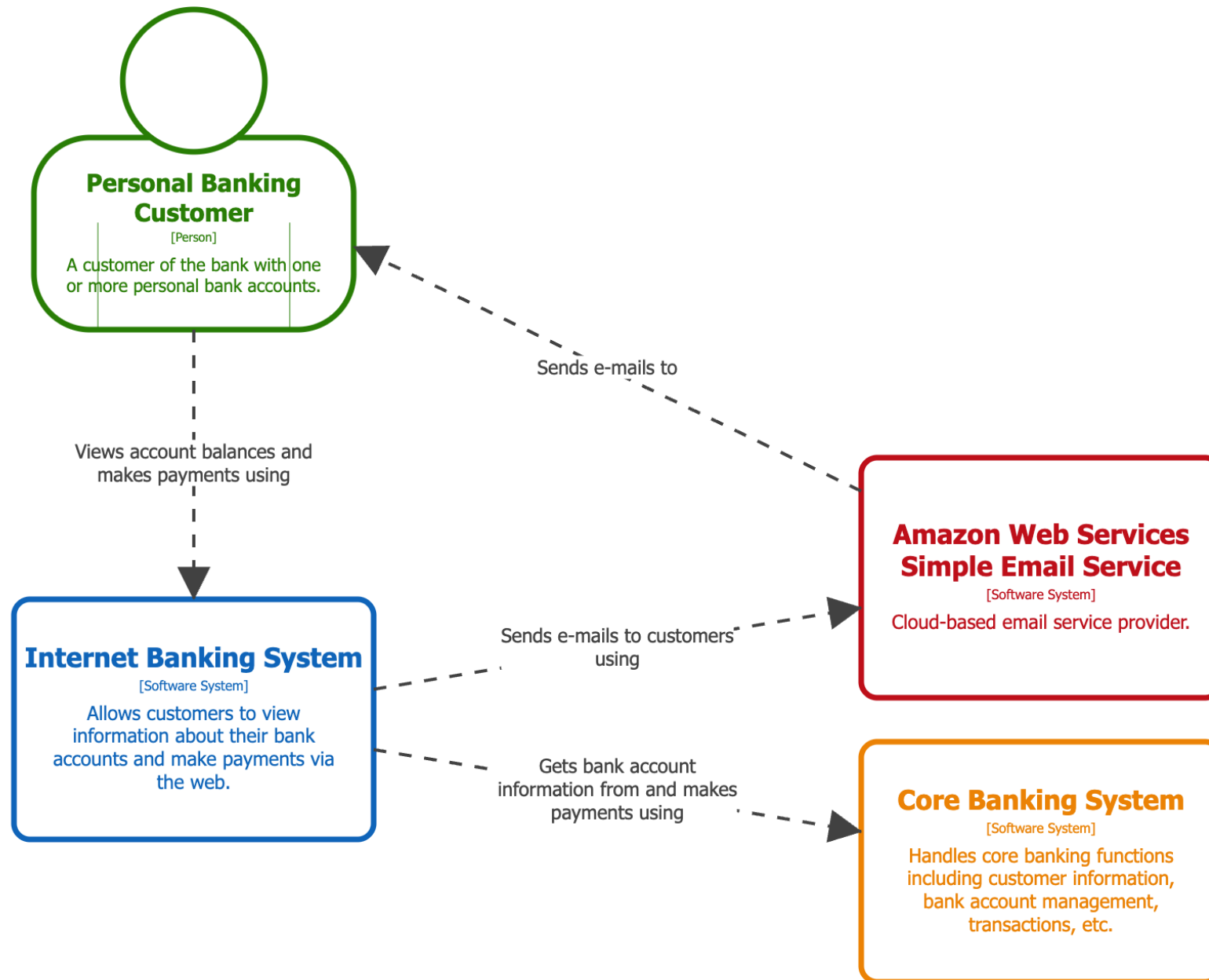
 **Level 3: Component**

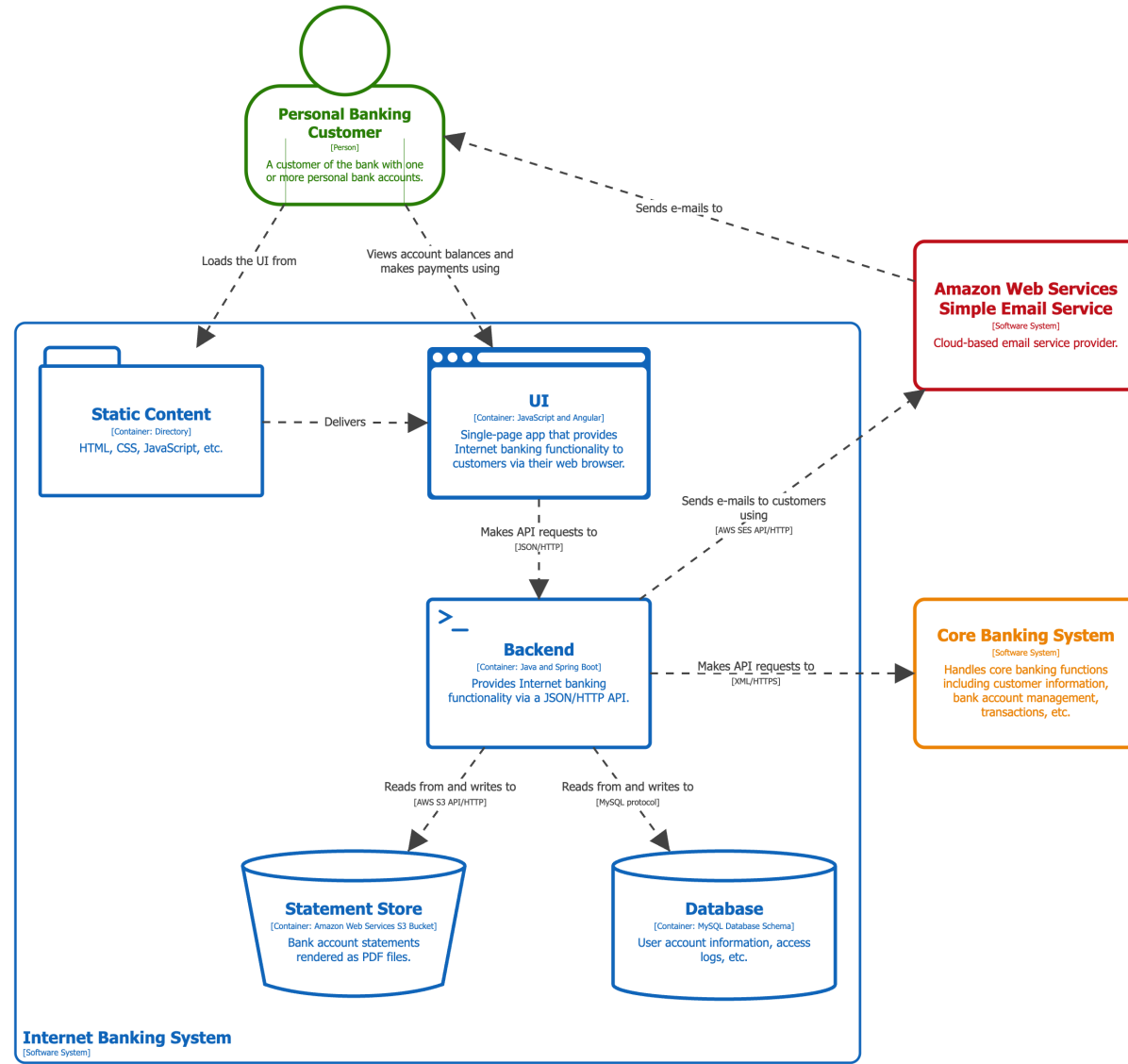
 **Level 4: Code**

C4 attempts to bring clarity and standardization... keeping your diagrams from bloating and being over-engineered.

Tại sao sử dụng C4 Model?

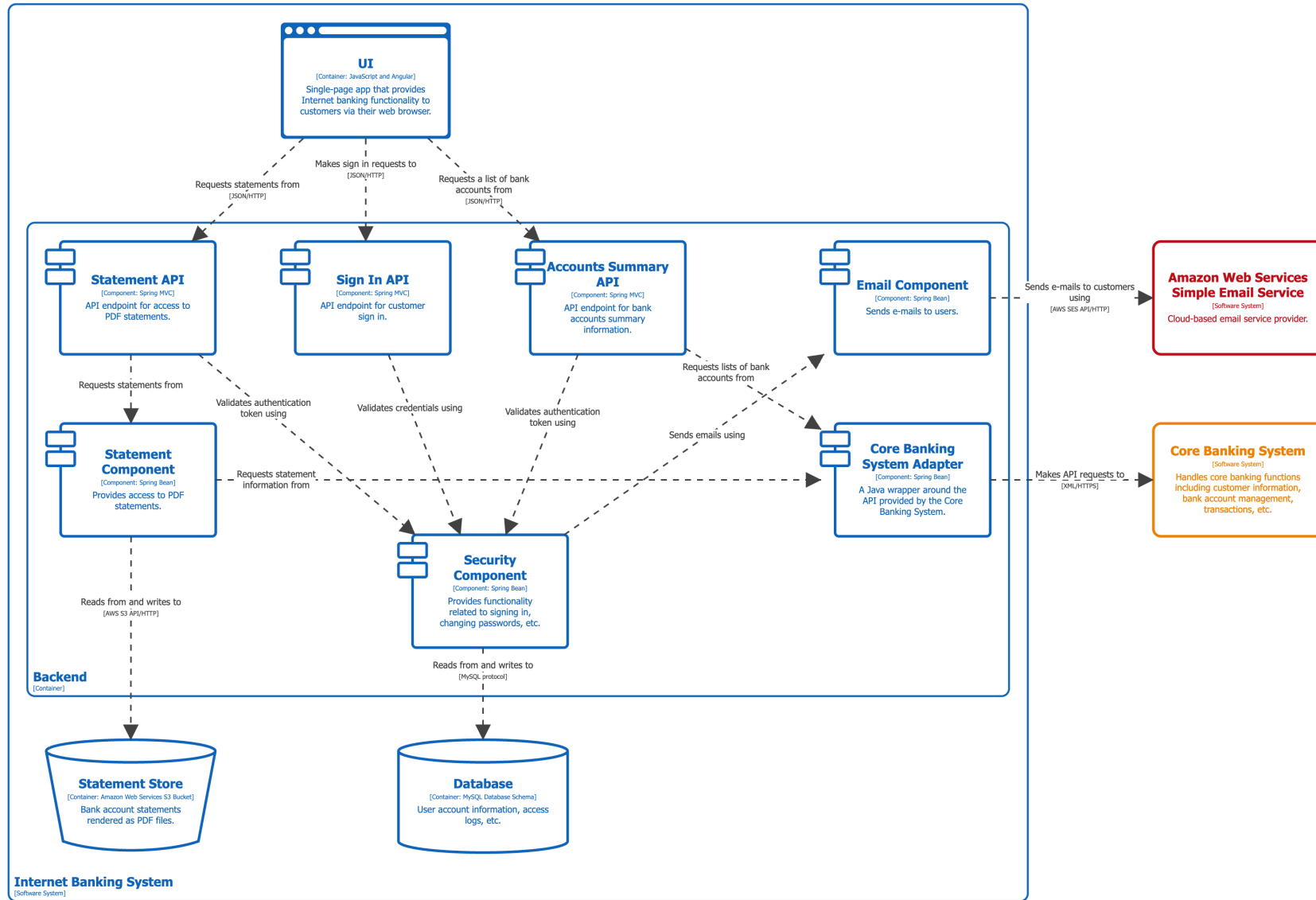
- ☕ **Trực quan hóa đa chiều:** Giúp các stakeholders khác nhau (Technical & Non-technical) hiểu hệ thống ở mức độ phù hợp.
- ☕ **Chuẩn hóa (Standardized):** Tránh sự mơ hồ của các ký hiệu tự phát.
- ☕ **Linh hoạt (Flexible):** Không bắt buộc dùng cả 4 level. Thường chỉ cần Context và Container là đủ.
- ☕ **Dễ bảo trì:** So với UML vốn dư thừa chi tiết, các diagram cấp cao của C4 ít thay đổi hơn và dễ cập nhật hơn.



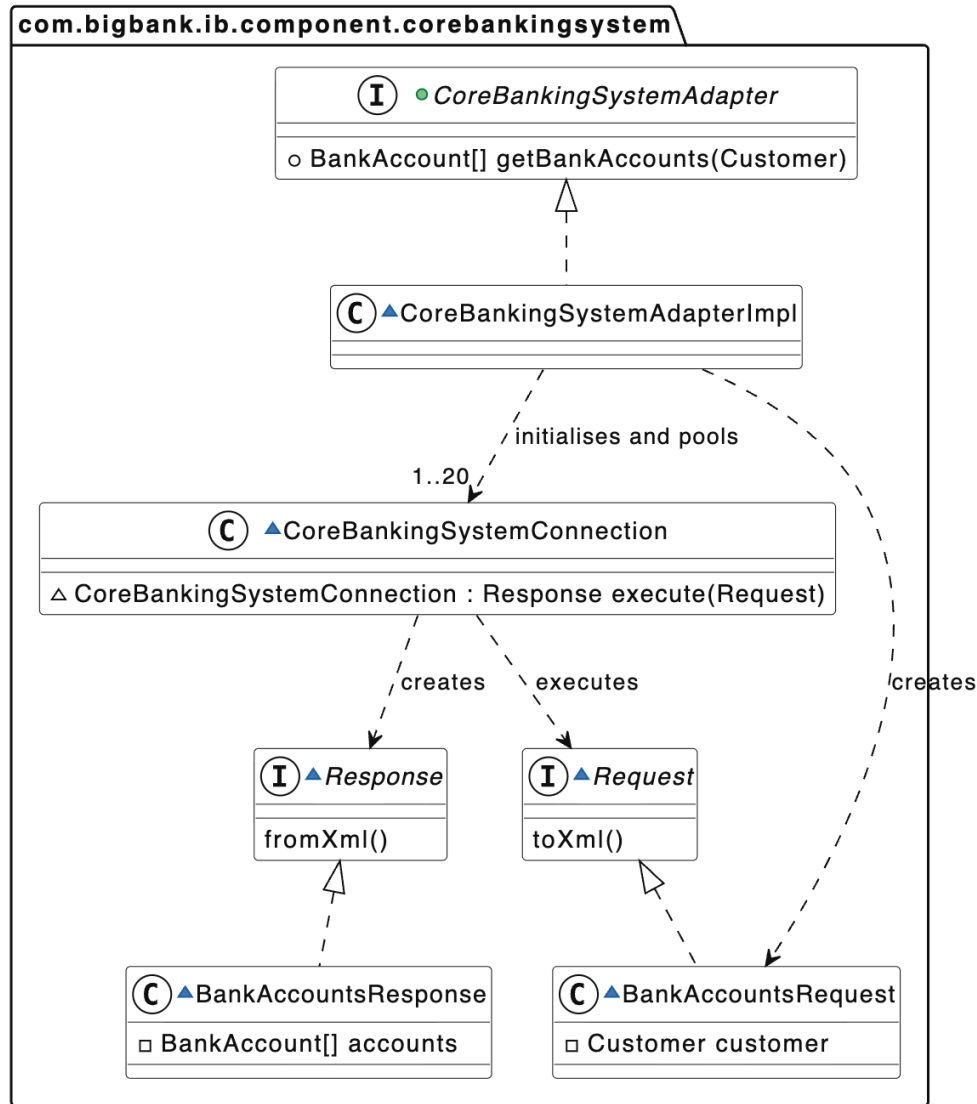


Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0



Component View: Internet Banking System - Backend
The component diagram for the Internet Banking System Backend | Simon Brown | c4model.com | License: CC BY 4.0



Code View: Internet Banking System - Backend - Core Banking System
 A summary of the implementation details for the Core Banking System Adapter component | Simon Brown |

Level 1: System Context Diagram

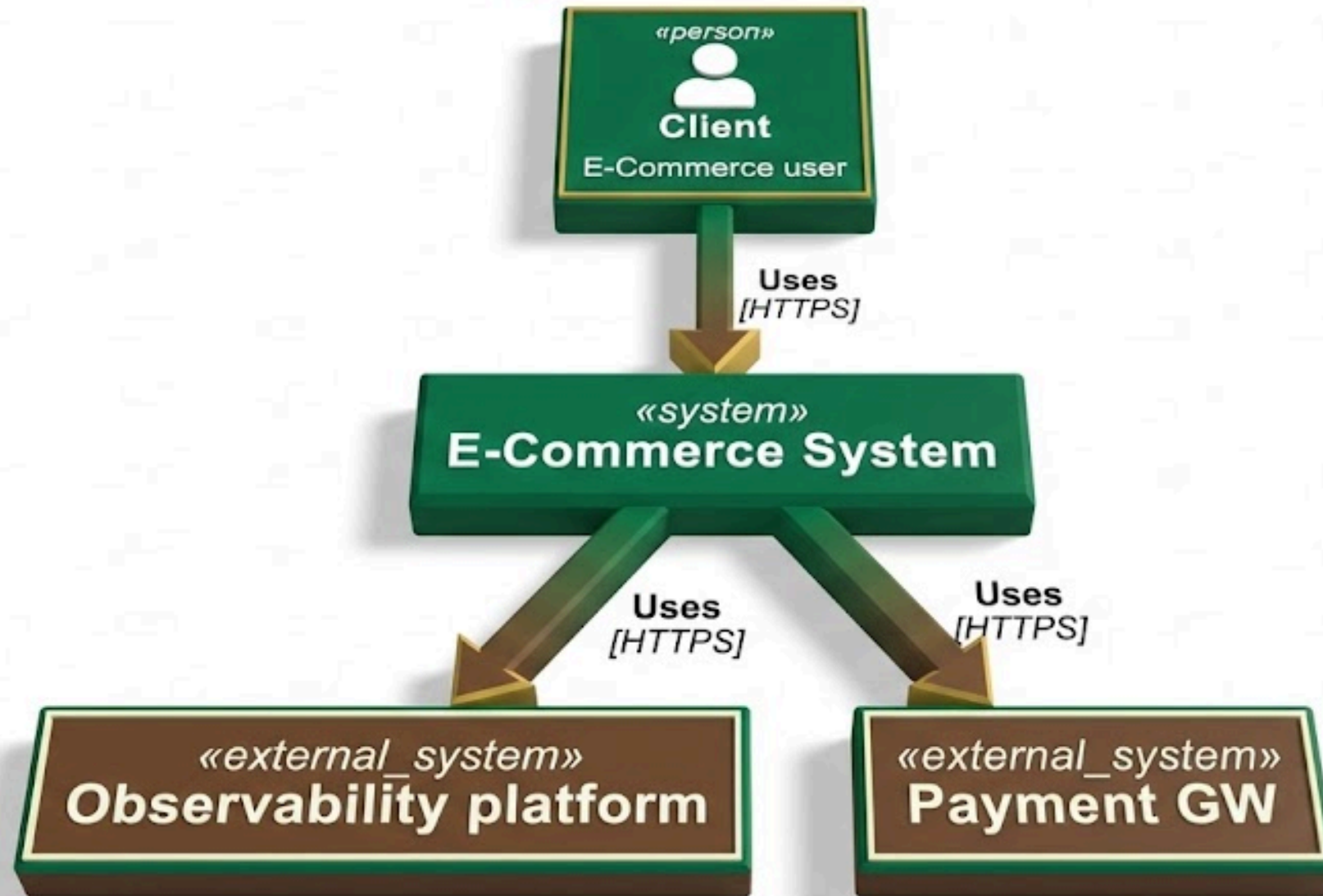
Mục đích:

- ☕ Cung cấp cái nhìn **tổng quan nhất** về hệ thống.
- ☕ Thể hiện hệ thống trong mối quan hệ với **người dùng** và **các hệ thống khác**.
- ☕ Không đi sâu vào chi tiết kỹ thuật.

Đối tượng:

- ☕ Tất cả mọi người (Technical, QA, BA, Product Owner, Business Stakeholders).

My E-Commerce



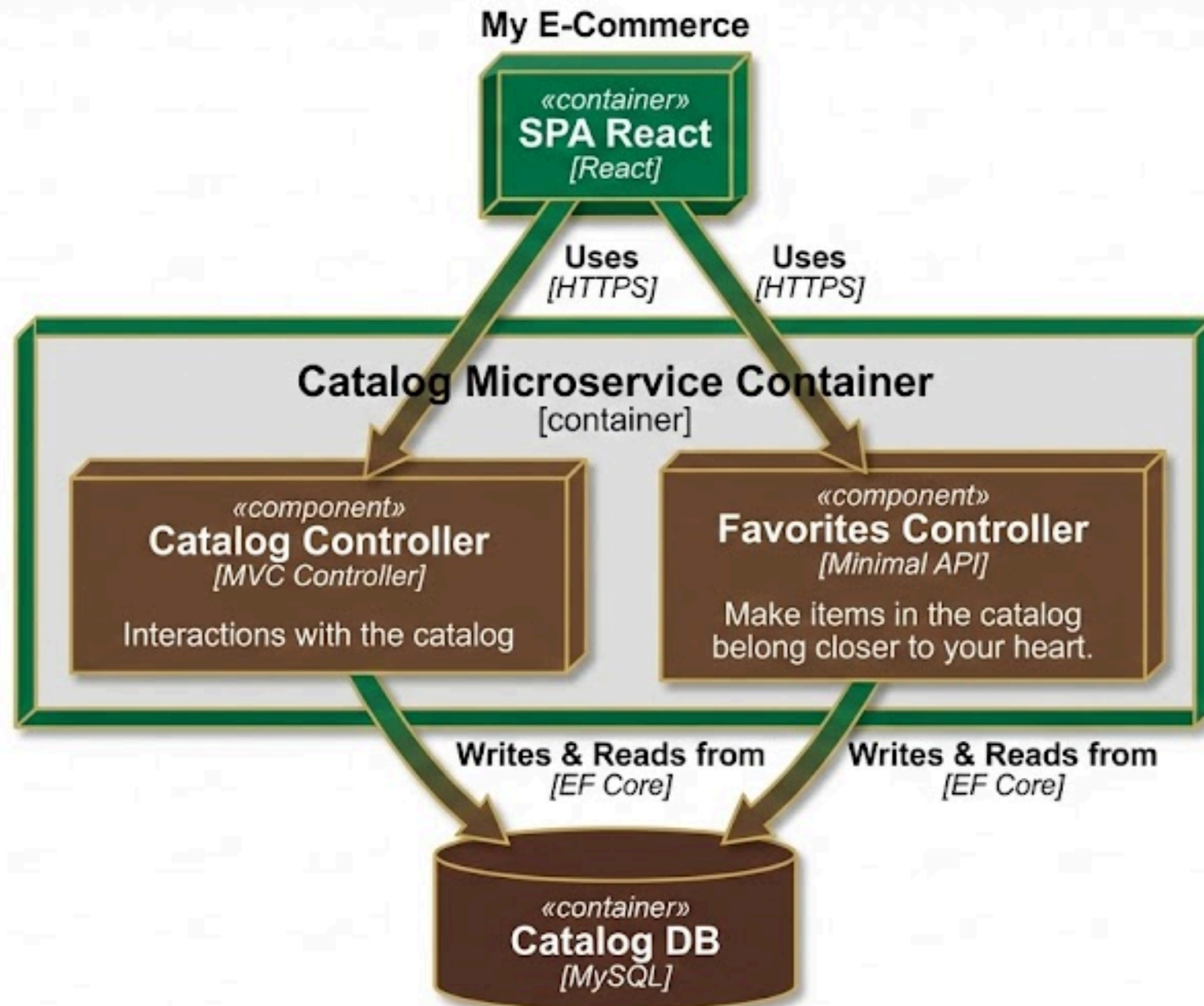
Level 2: Container Diagram

Mục đích:

- ☕ **Zoom in** vào hệ thống để thấy các **Container** (đơn vị triển khai độc lập).
- ☕ Container có thể là: Web App, Mobile App, API Application, Database, File System, Microservice.
- ☕ Thể hiện công nghệ sử dụng (Java, React, SQL...) và giao thức giao tiếp (HTTPS, JDBC...).

Đối tượng:

- ☕ Technical Stakeholders (Developers, Architects, Ops).



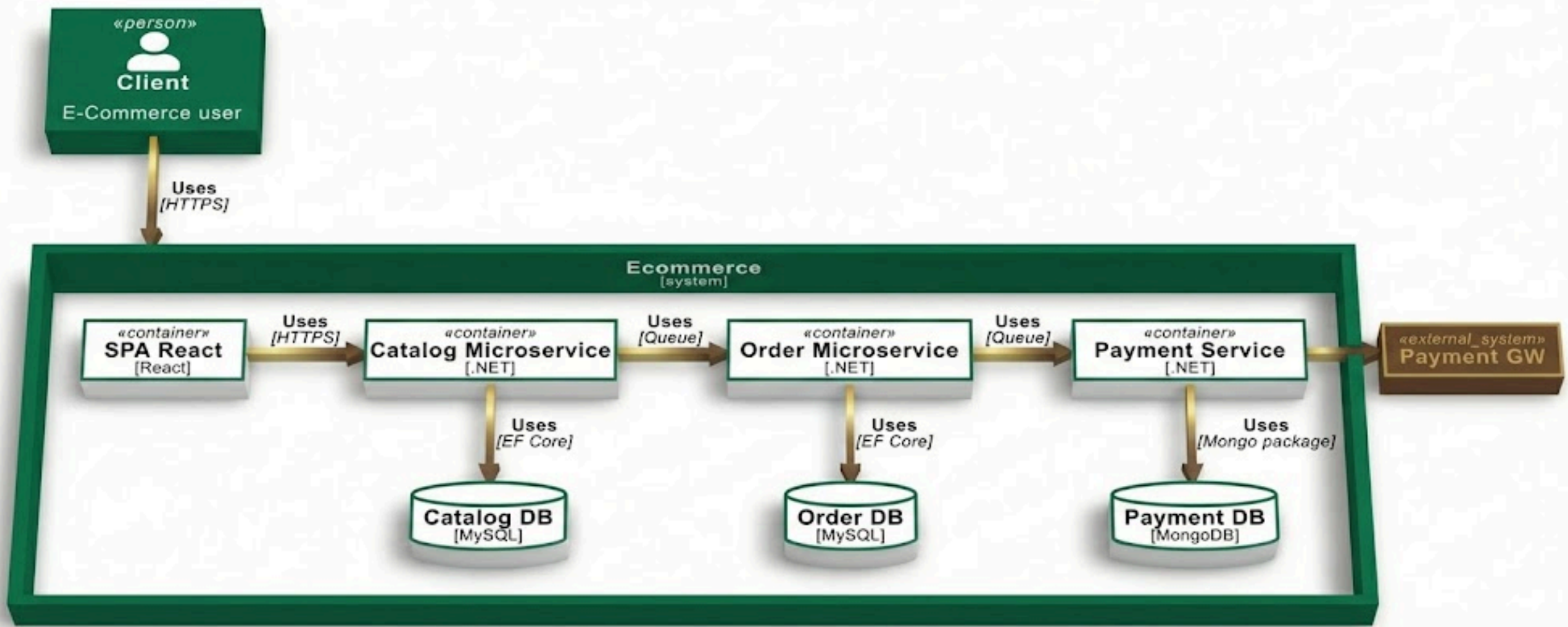
Level 3: Component Diagram

Mục đích:

- ☕ **Zoom in** vào một Container cụ thể để thấy các **Component** bên trong.
- ☕ Component là nhóm các chức năng/class liên quan (Controller, Service, Repository...).
- ☕ Thể hiện cấu trúc nội bộ của ứng dụng.

Đối tượng:

- ☕ Developers (Đội ngũ phát triển trực tiếp hệ thống đó).



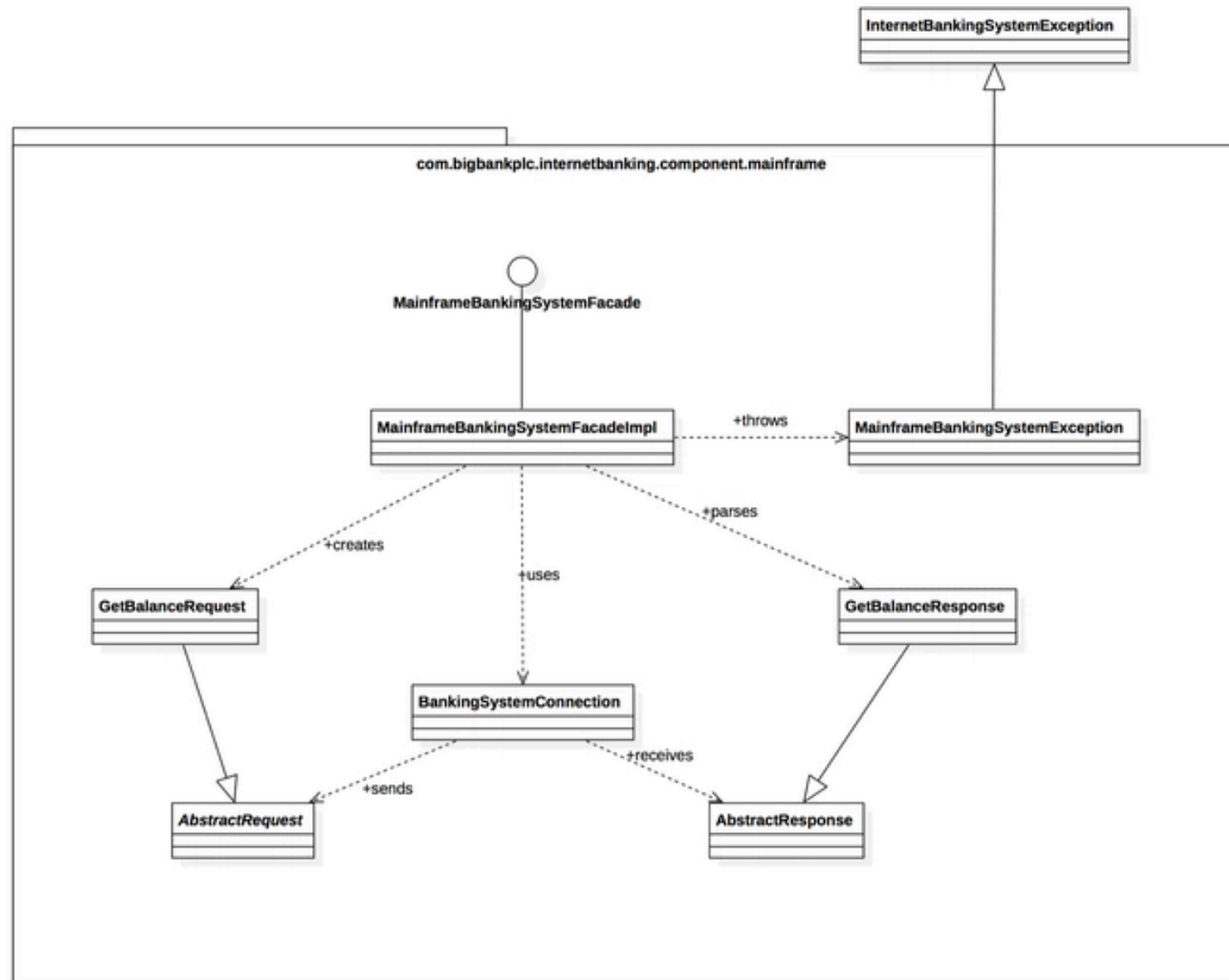
Level 4: Code Diagram

Mục đích:

- ☕ **Zoom in** vào một Component để thấy chi tiết **Class/Interface** (thường dùng UML Class Diagram).
- ☕ Thể hiện chi tiết thực thi.

Lưu ý:

- ☕ **Không khuyến khích** vẽ thủ công vì rất dễ lỗi thời và tốn công bảo trì.
- ☕ Nên được **generate** tự động từ IDE hoặc tools.
- ☕ Chỉ nên dùng cho những phần logic cực kỳ phức tạp cần giải thích rõ.



Khi nào sử dụng level nào?

- ☕ Luôn bắt đầu với **Level 1 (Context)** và **Level 2 (Container)**. Đây là 2 diagram quan trọng nhất và mang lại nhiều giá trị nhất.
- ☕ Sử dụng **Level 3 (Component)** khi hệ thống rất phức tạp hoặc để document cho team dev mới.
- ☕ Hạn chế sử dụng **Level 4 (Code)** trừ khi thực sự cần thiết.

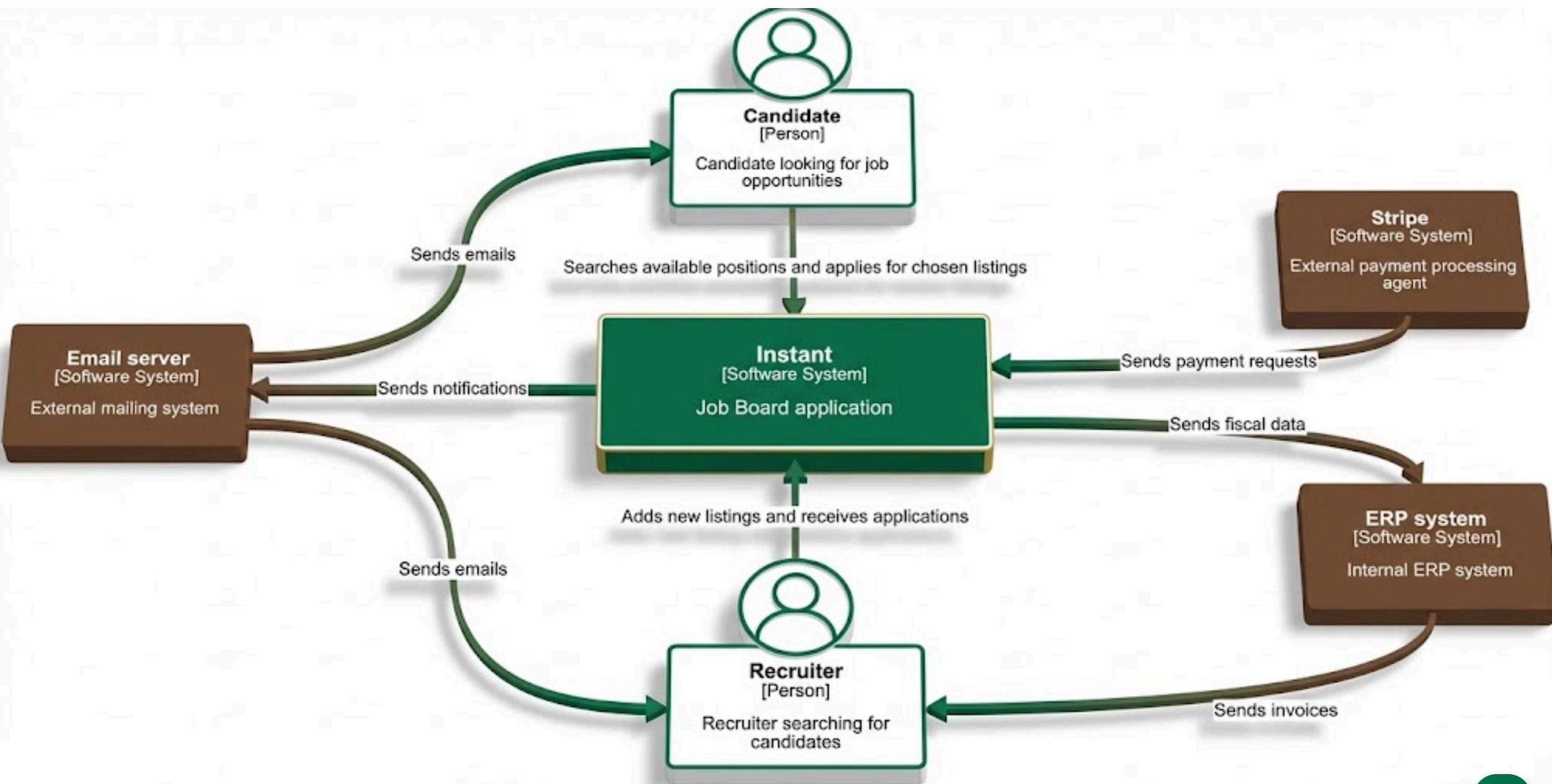
Architecture is basically concerned with higher-level decisions... design covers the implementation.

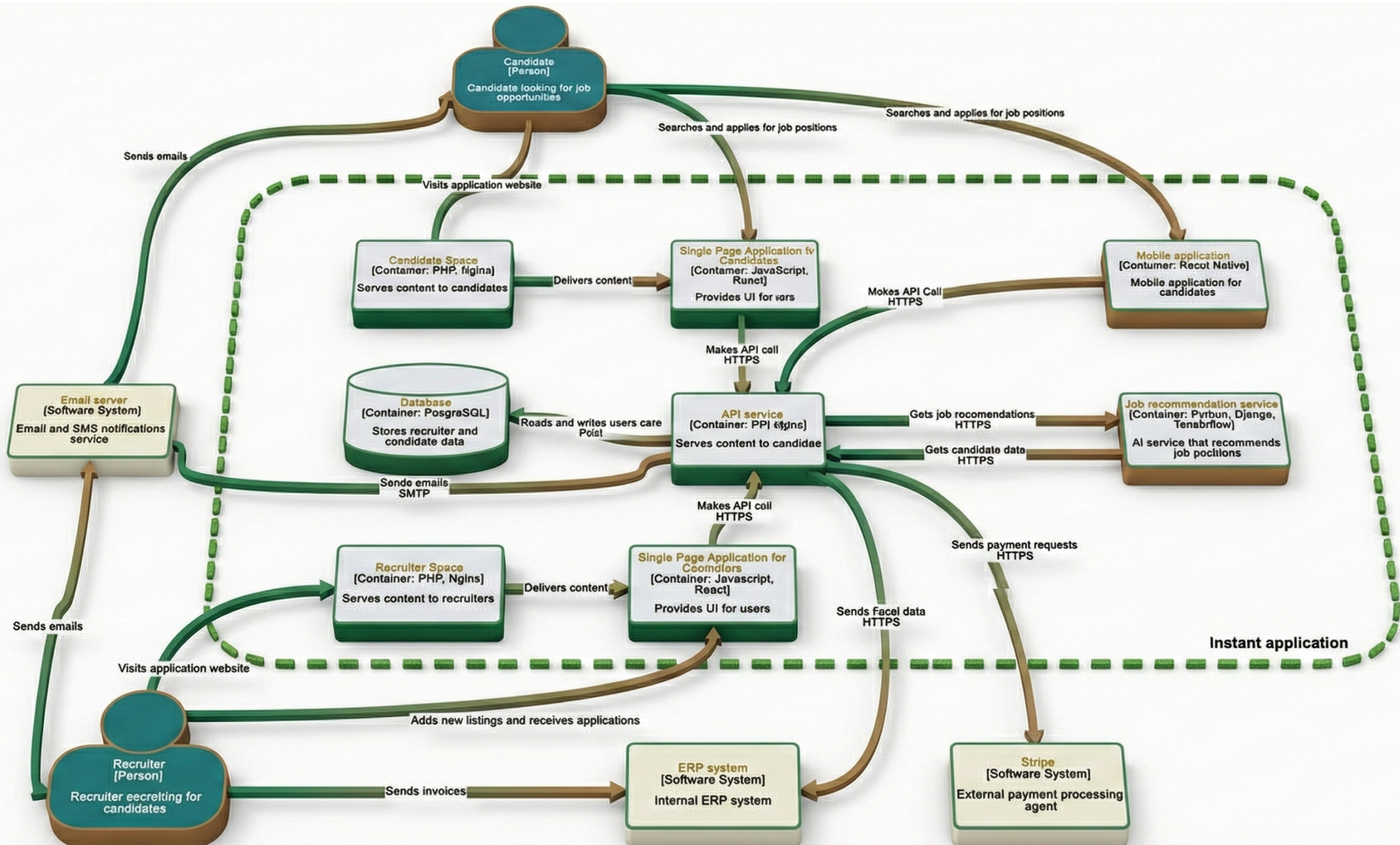
Công cụ hỗ trợ

Thay vì vẽ tay, bạn có thể sử dụng **Diagram as Code**:

- ☕ **C4-PlantUML**: Extension cho PlantUML.
- ☕ **Mermaid JS**: Hỗ trợ vẽ C4 (thông qua syntax graph/class hoặc plugin).
- ☕ **Structurizr**: Tool chuyên dụng cho C4 Model của Simon Brown.
- ☕ **Draw.io**: Có bộ thư viện icon C4.

Case Study: Hệ thống "Instant"



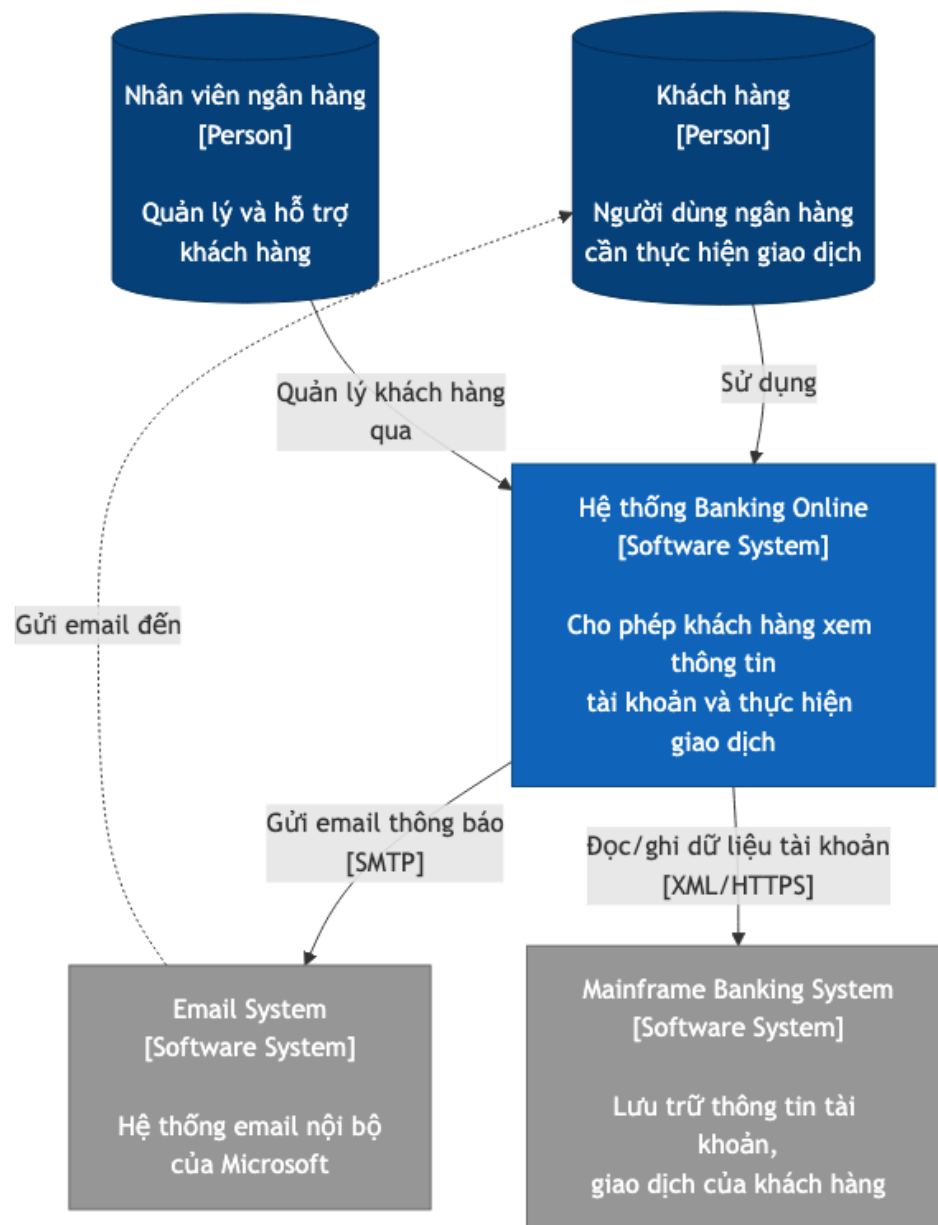


Case Study: Hệ thống Banking Online

Level 1: System Context

Mô tả:

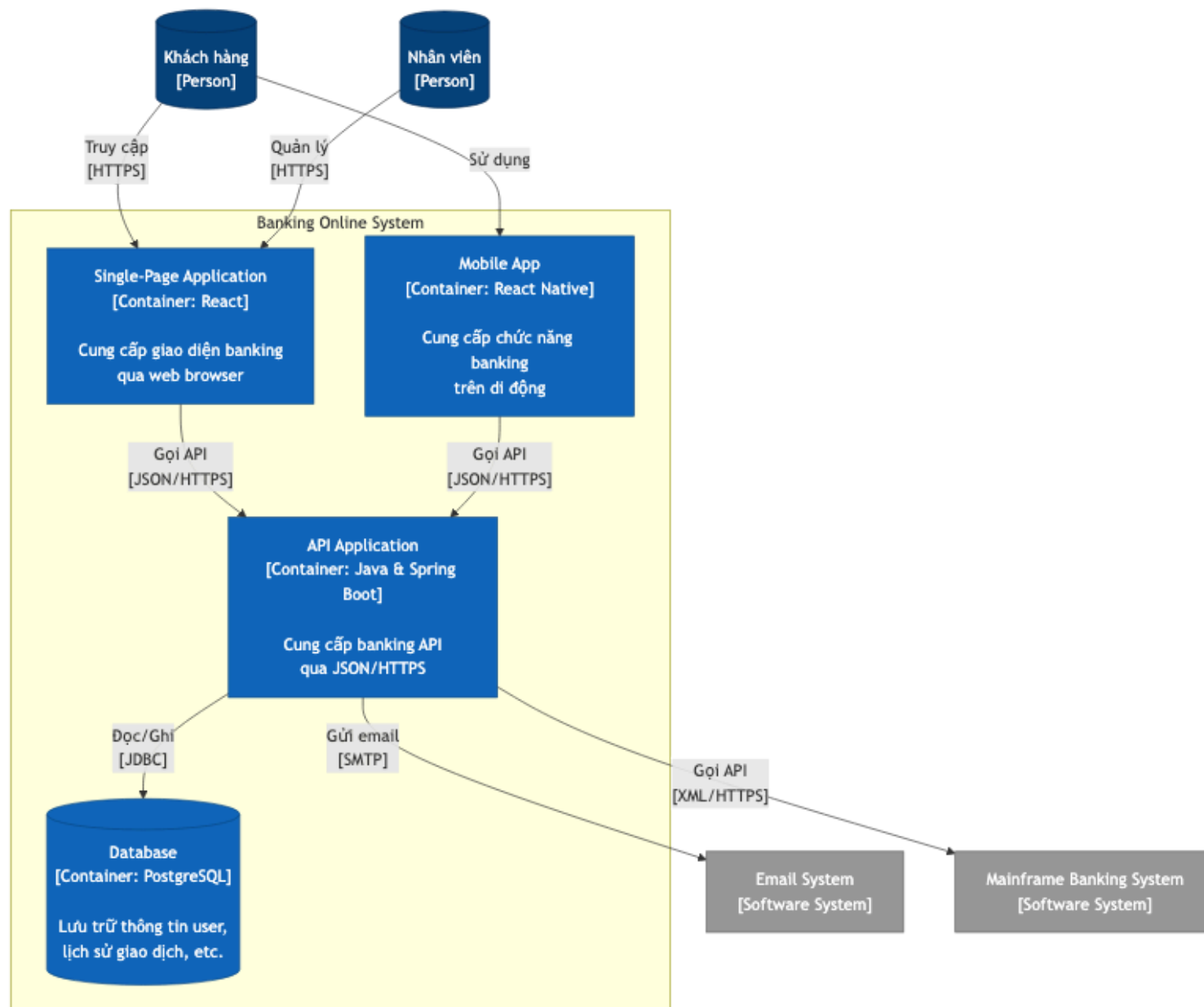
-  Hệ thống Banking Online cho phép khách hàng thực hiện các giao dịch ngân hàng, xem số dư, chuyển tiền.
-  Hệ thống tích hợp với **Email Service** để gửi thông báo và **Mainframe Banking System** để xử lý giao dịch thực tế.



Level 2: Container

Hệ thống gồm có:

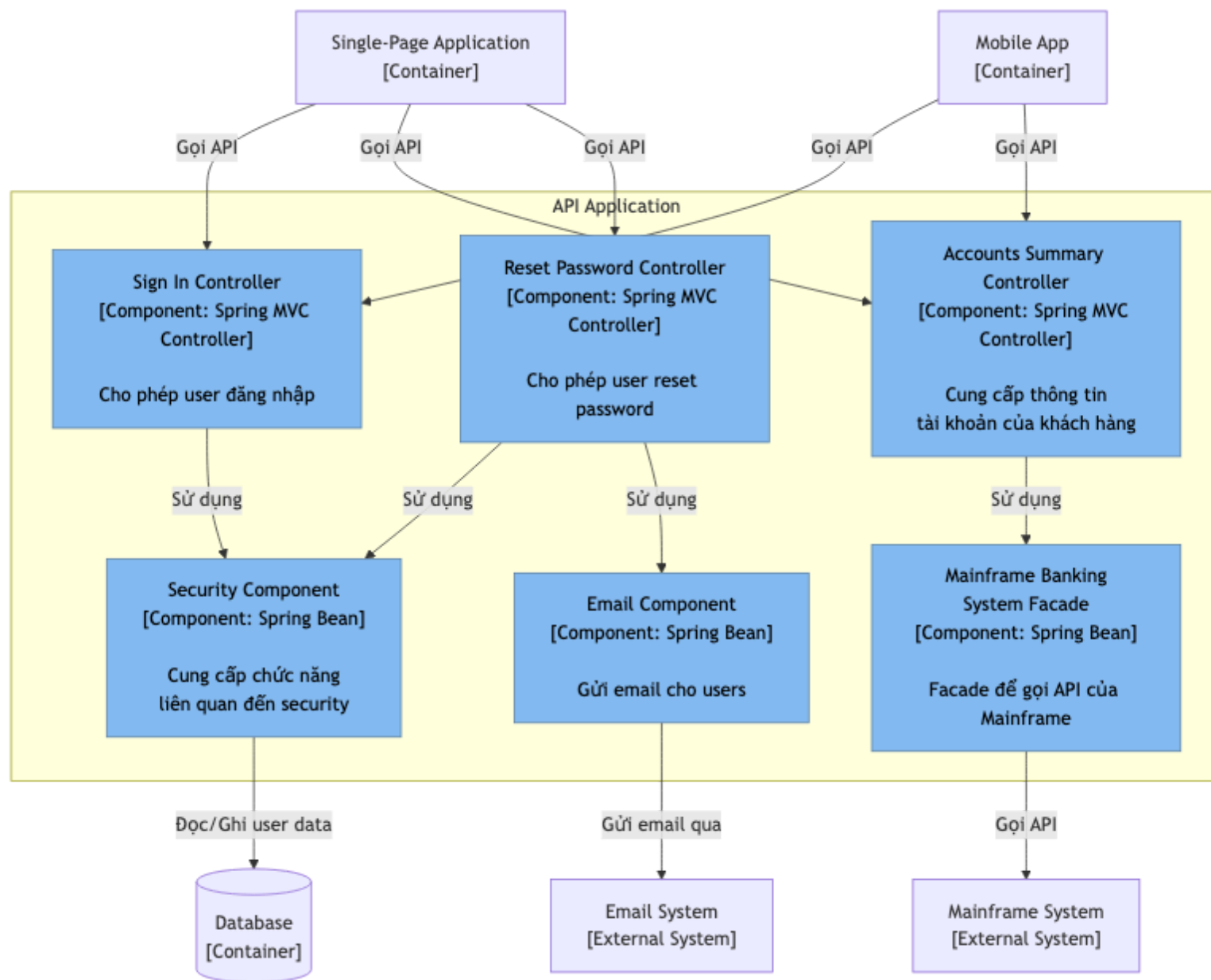
-  **Single-Page Application (SPA):** Giao diện web React chạy trên browser
-  **Mobile App:** Ứng dụng di động native
-  **API Application:** Backend xử lý business logic (Java/Spring Boot)
-  **Database:** PostgreSQL lưu trữ dữ liệu

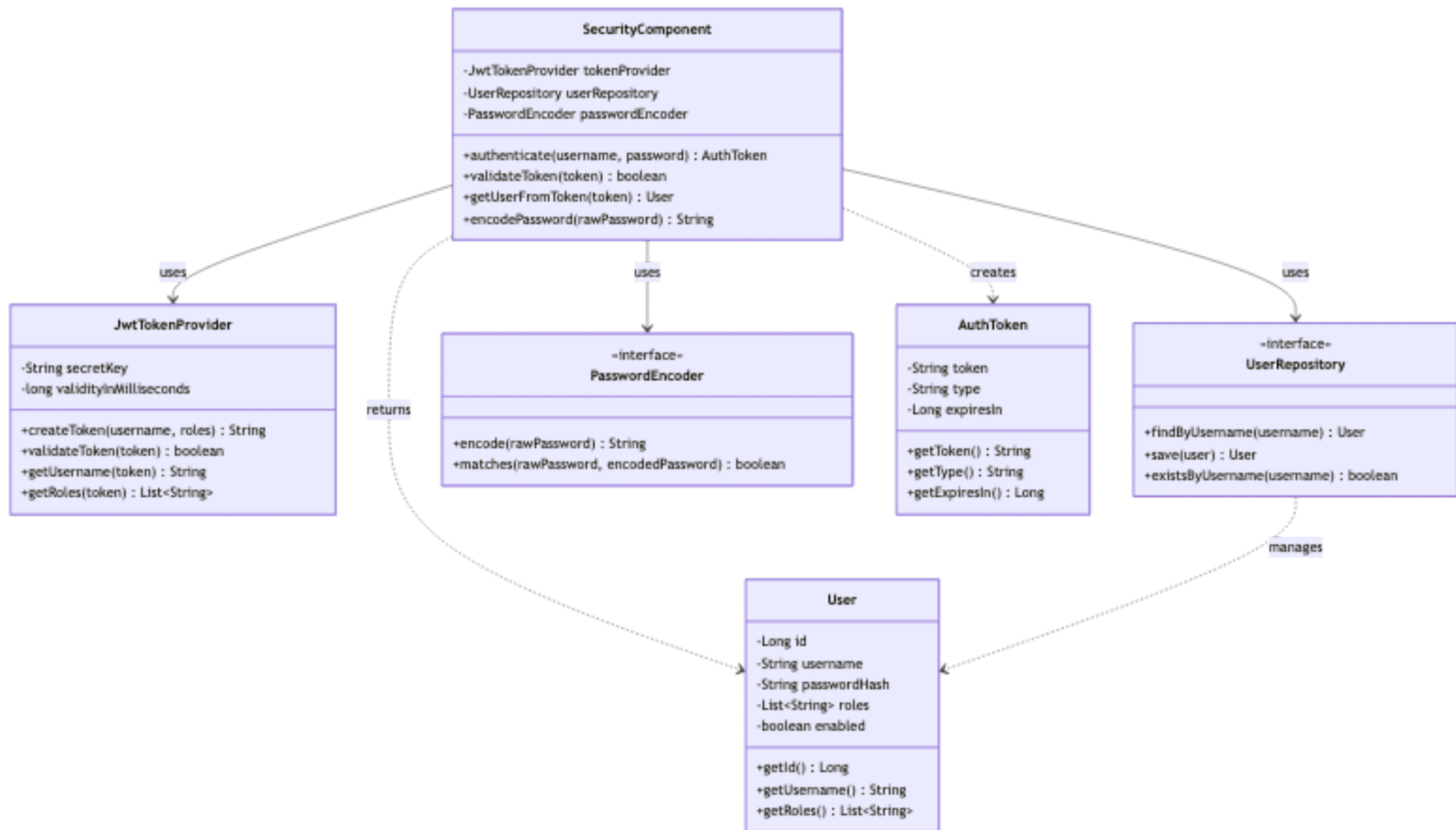


Level 3: Component

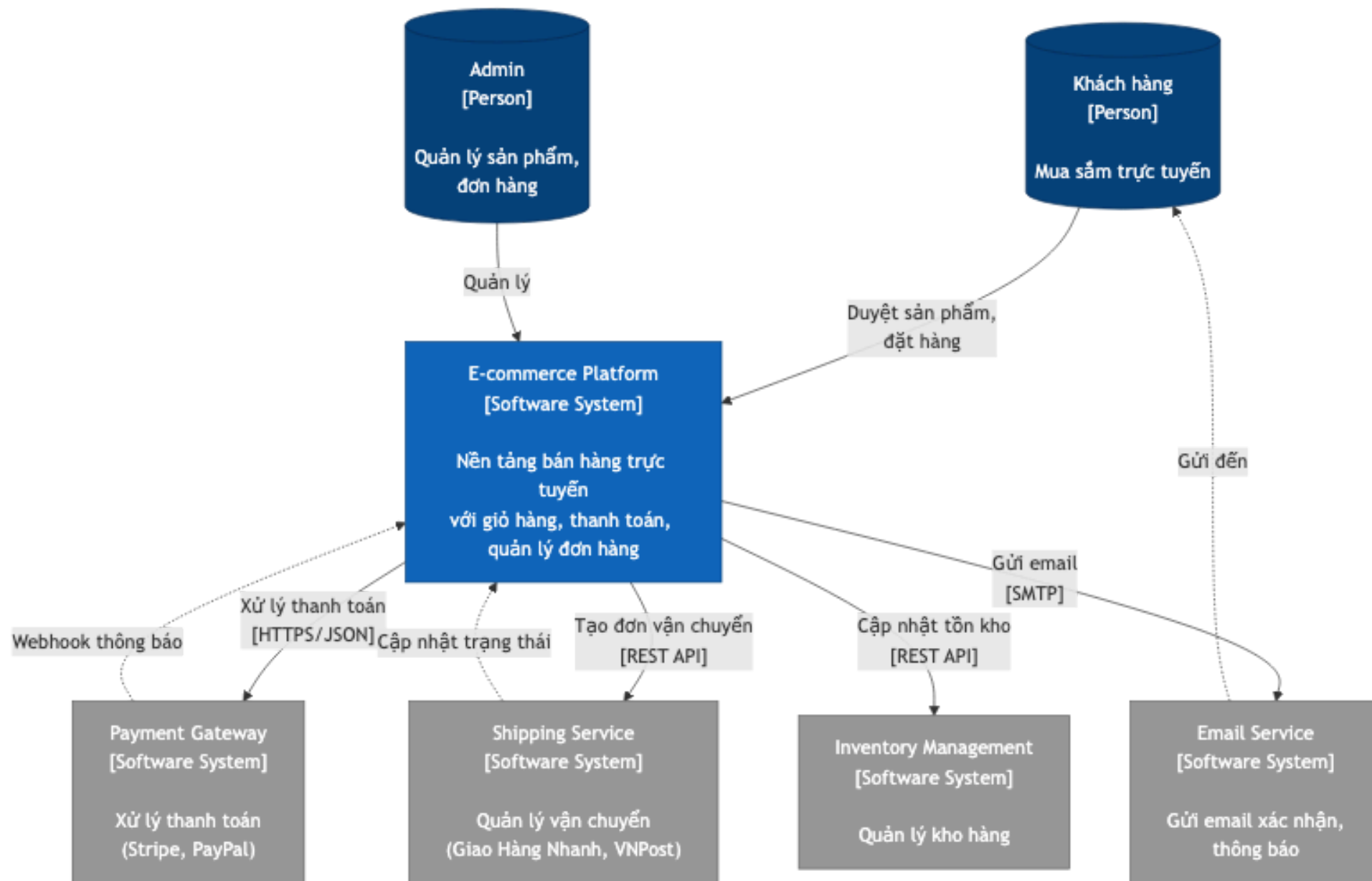
Zoom vào API Application để thấy các component bên trong:

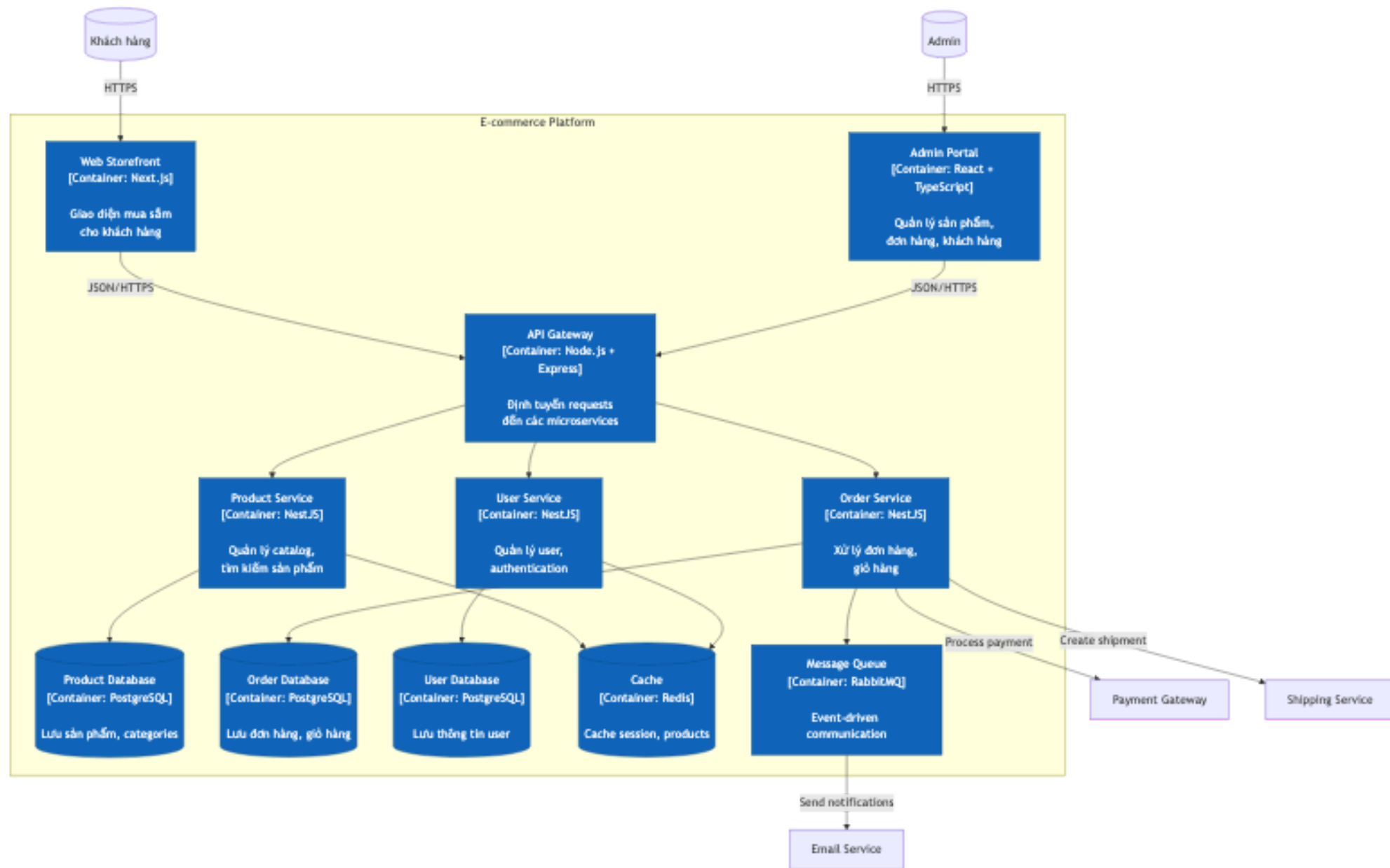
-  **Sign In Controller:** Xử lý authentication
-  **Accounts Summary Controller:** Cung cấp tóm tắt tài khoản
-  **Reset Password Controller:** Xử lý reset mật khẩu
-  **Security Component:** Xử lý JWT, authorization
-  **Mainframe Facade:** Gateway tới Mainframe System
-  **Email Component:** Gateway tới Email System





Case Study: E-commerce Platform





So sánh các công cụ vẽ C4

Công cụ	Ưu điểm	Nhược điểm	Phù hợp với
Mermaid	- Syntax đơn giản - Tích hợp GitHub/GitLab - Miễn phí	- Hạn chế về styling - Không có C4 syntax native	Developer muốn version control diagrams
Structurizr	- Chuyên dụng cho C4 - DSL mạnh mẽ - Auto-layout tốt	- Có phí cho cloud version - Learning curve	Architect muốn tool chuyên nghiệp

So sánh các công cụ vẽ C4

Công cụ	Ưu điểm	Nhược điểm	Phù hợp với
PlantUML + C4	- C4 syntax native - Nhiều tùy chỉnh - Miễn phí	- Syntax phức tạp - Setup phức tạp hơn	Team đã quen PlantUML
Draw.io	- WYSIWYG - Dễ dùng - Miễn phí	- Khó version control - Thiếu automation	Non-technical stakeholders

Kết luận

C4 Model là công cụ mạnh mẽ để **tài liệu hóa kiến trúc** một cách hiệu quả, giúp thu hẹp khoảng cách giữa mô hình trừu tượng và code thực tế.

Các takeaways quan trọng:

- ☕ Bắt đầu từ **Context và Container** - Đây là 2 level mang lại giá trị cao nhất
- ☕ **Diagram as Code** - Sử dụng Mermaid, PlantUML để version control diagrams
- ☕ **Keep it updated** - Diagram chỉ có giá trị khi được cập nhật thường xuyên
- ☕ **Phù hợp với audience** - Sử dụng level phù hợp cho từng đối tượng
- ☕ **Không over-engineer** - Chỉ vẽ những gì thực sự cần thiết

Tài liệu tham khảo

-  C4 Model
-  Structurizr
-  PlantUML
-  Mermaid